Lesson 3: Spatial Analysis

Unlocking Spatial Patterns in Transcriptomics Data

Lesson Overview - Today's Journey

- Section 1: Introduction to Spatial Analysis
- Section 2: Spatially Variable Genes - Moran's I
- Section 3: Spatial Gene Modules - Hotspot
- Section 4: Neighborhood & Co-occurrence - Squidpy
- Section 5: Spatial Domains & Communities
- Section 6: Ligand-Receptor Interactions
- Section 7: Advanced Spatial Patterns
- Section 8: Integration & Visualization
- Plus: Hands-on exercises and Q&A

Learning Objectives

- Identify spatially variable genes and patterns
- Detect spatial domains and cellular communities
- Analyze cell-cell interactions in spatial context
- Apply spatial statistics to biological interpretation
- Integrate multiple spatial analysis approaches
- Visualize and communicate spatial findings effectively

Introduction to Spatial Analysis

Why Spatial Context Matters

Why Spatial Context Matters in Transcriptomics

- Traditional scRNA-seq loses spatial organization
- Cells function in specific tissue microenvironments
- Spatial patterns reveal:
  - Tissue architecture and organization
  - Cell-cell communication networks
  - Functional tissue domains
  - Disease progression patterns
- Example: Tumor cells at edge vs. core have different behaviors

Overview of Spatial Analysis Approaches

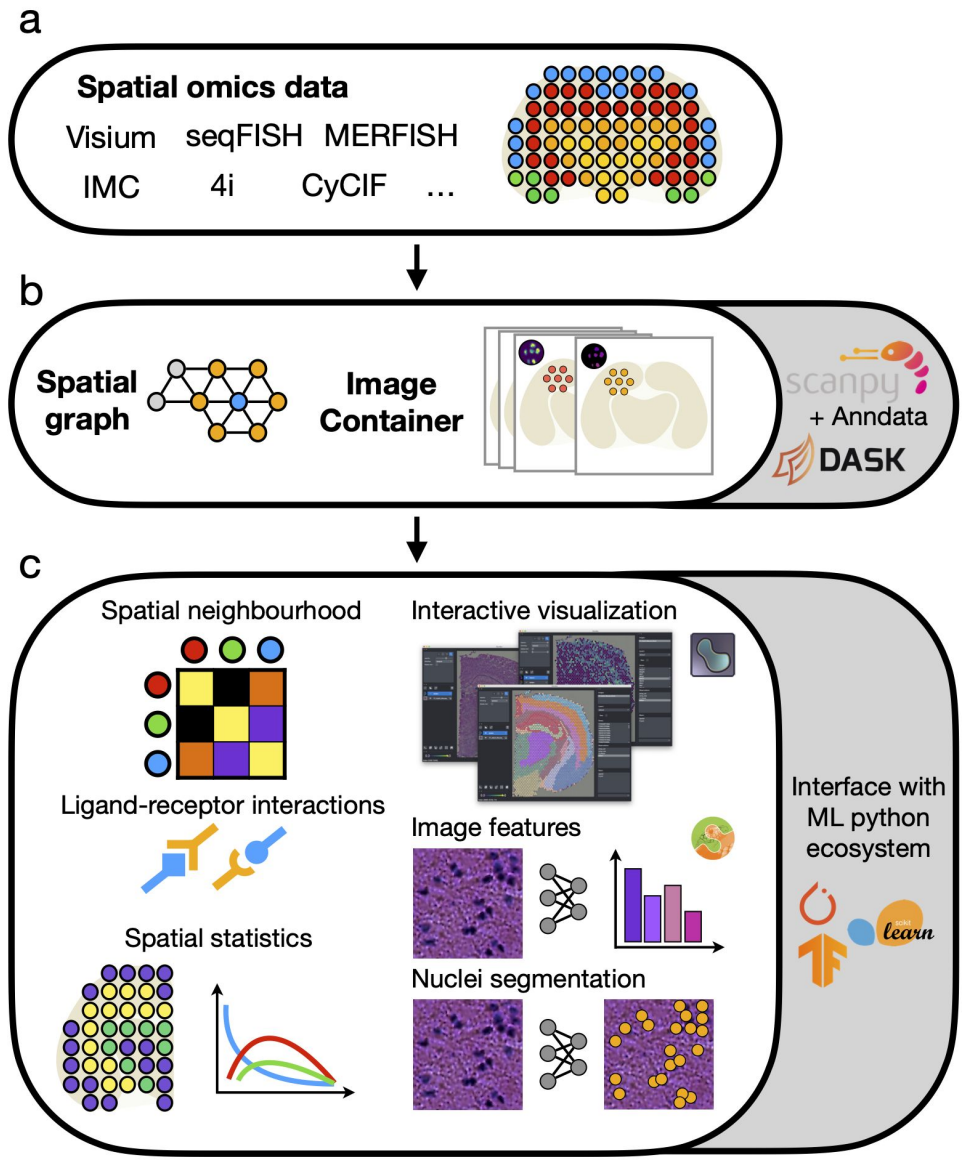
- Gene-centric: Spatially variable genes (Moran's I, SpatialDE)
- Module-centric: Coherent gene modules (Hotspot)
- Cell-centric: Neighborhood and co-occurrence (Squidpy)
- Domain-centric: Spatial domains (Banksy, GraphST)
- Interaction-centric: Cell communication (CellPhoneDB)
- Each approach provides complementary insights
- Integration gives the complete picture

Key Spatial Concepts - Foundation Terms

- Spatial Autocorrelation: Nearby locations tend to have similar values
  - Tobler's First Law: 'Everything is related, but near things more so'
- Spatial Neighborhoods: Cells within defined distance or graph connection
- Spatial Domains: Regions with distinct molecular signatures
- Spatial Patterns: Gradients, hotspots, clusters, boundaries
- Spatial Statistics: Quantitative measures of spatial relationships

Squidpy

- Python library for the analysis of spatial omics data, built on top of Scanpy and the AnnData data structure.
- Spatial graph construction - Builds neighborhood graphs based on physical coordinates (kNN, radius-based).
- Enables spatial statistics such as Moran's I and Geary's C.
- Identifies genes with significant spatial structure.
- Neighborhood enrichment
- Image integration - Links high-resolution tissue images with molecular data.
- Ligand–receptor analysis in space

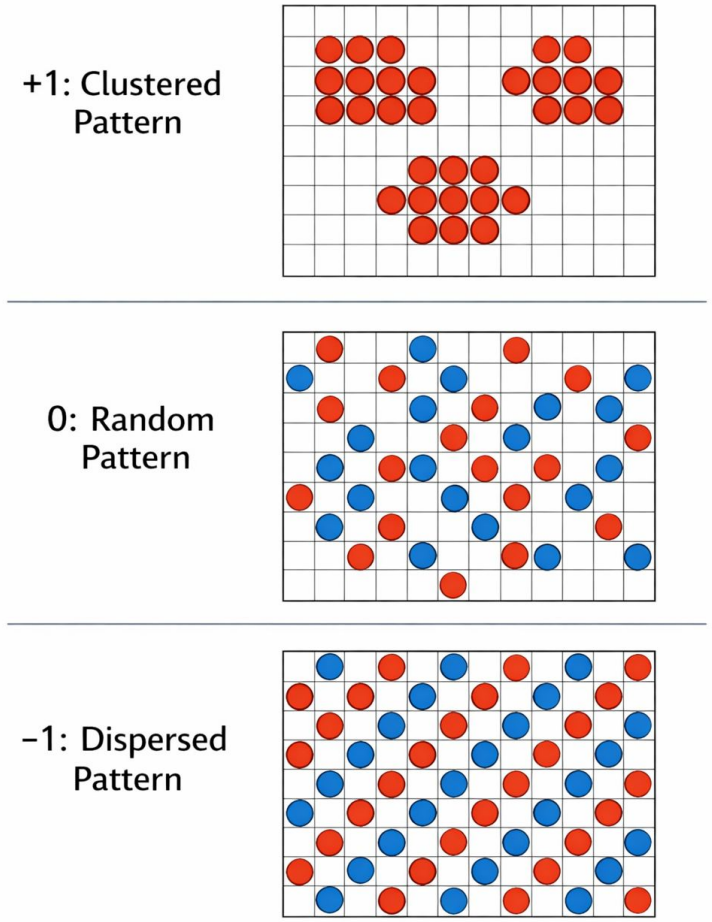


Spatially Variable Genes

Identifying Genes with Spatial Patterns

Moran's I Spatial Autocorrelation

- Quantifies how gene expression correlates with spatial location
- Global Moran's I: Overall spatial autocorrelation (-1 to +1)
- +1 = Perfect positive autocorrelation (clustered expression)
- 0 = Random spatial pattern (no autocorrelation)
- -1 = Perfect negative autocorrelation (dispersed pattern)
- Statistical significance via permutation testing



Understanding Moran's I - The Math

- Formula: $I = (N/W) \times \Sigma(w_{ij}(x_i - \bar{x})(x_j - \bar{x})) / \Sigma(x_i - \bar{x})^2$
  - N: Number of spatial units (spots/cells)
  - W: Sum of all spatial weights from entire spatial adjacency matrix
  - w_{ij}: Spatial weight between location i and j
  - x_i: Expression value at location i
  - $\bar{x}$: Mean expression across all locations
- Looks scary but concept is simple: 'Are neighbors similar?'
- Software does the calculation - you interpret the result!

Computing Moran's I with Squidpy

Identify and visualize spatially variable genes

```
import squidpy as sq
import scanpy as sc
# Load spatial data
adata = sc.read_h5ad('spatial_data.h5ad')
# Build spatial neighbors graph
sq.gr.spatial_neighbors(adata, coord_type='grid')

# Calculate Moran's I for all genes
sq.gr.spatial_autocorr(
    adata,
    mode='moran',
    n_perms=100,  # Number of permutations for p-value when calculating Moran’s I
    n_jobs=8
)
# Results stored in adata.uns['moranI']
# Filter spatially variable genes (FDR < 0.05)
sv_genes = adata.uns['moranI'][
    adata.uns['moranI']['pval_norm_fdr_bh'] < 0.05
]
# Visualize top gene
top_gene = sv_genes.iloc[0].name
sq.pl.spatial_scatter(adata, color=top_gene)
```

Interpreting Spatially Variable Genes

- High Moran's I genes show spatial structure:
  - Anatomical zones (e.g., cortex vs medulla)
  - Disease progression (tumor center vs edge)
  - Developmental gradients (anterior-posterior axis)
  - Microenvironment niches (hypoxic, inflamed)
- Low/zero Moran's I genes: housekeeping or stochastic
- Biological validation is essential!
- Compare with known markers and pathways

SpatialDE: Statistical Framework for SVGs

- Alternative to Moran's I for spatially variable genes
- Model-based approach using Gaussian processes
- Advantages:
  - Models spatial length scales (how far patterns extend)
  - Tests multiple pattern types (periodic, linear)
  - Provides interpretable parameters
- More computationally intensive than Moran's I
- Best for datasets <5000 spots (scalability)
- Widely used in spatial transcriptomics literature

Comparing Spatially Variable Gene Methods

Moran's I

- Fast and scalable (100K+ observations)
- Simple interpretation (one value)
- Global spatial autocorrelation
- Non-parametric (no model assumptions)
- Widely implemented (Squidpy, Giotto)
- Best for initial screening

SpatialDE

- Model-based (Gaussian processes)
- Characterizes pattern types
- Estimates spatial length scales
- More interpretable parameters
- Slower (best for <5K spots)
- Best for detailed pattern analysis

Alternative Method: Getis-Ord Gi* Statistic

- Detects local spatial hotspots and coldspots
- Differences from Moran's I:
  - Moran's I: global spatial autocorrelation (one value per gene)
  - Getis-Ord: local statistic (value per location - observation)
- Identifies specific regions with high/low expression clusters
- Z-scores indicate significance of local clustering
- Useful for finding localized expression patterns
- Complementary to Moran's I for complete picture

Spatial Gene Modules with Hotspot

Finding Coherent Gene Programs

Hotspot: Spatial Gene Modules

Identifies coherent spatial co-expression patterns

1. Compute spatial autocorrelation for each gene (similar to Moran's I)
2. Compute local correlations between genes, weighted by spatial proximity
3. Cluster genes based on their spatial correlation patterns
4. Identify modules of genes with coherent spatial variation

Particularly powerful for identifying functional programs

Hotspot Workflow: From Data to Modules

Complete pipeline for spatial module detection

```
import hotspot
import scanpy as sc
# Load spatial data
adata = sc.read_h5ad('spatial_data.h5ad')
# Create Hotspot object
hs = hotspot.Hotspot(
    adata,
    layer_key='counts',
    model='danb',  # Negative binomial for count data
    latent_obsm_key='X_pca'  # Use PCA for efficiency
)

# Compute spatial autocorrelation for all genes
hs.create_knn_graph(
    weighted_graph=False,
    n_neighbors=30  # Spatial neighborhood size
)

# Find autocorrelated genes
hs_results = hs.compute_autocorrelations(jobs=8)
```

```
# Select spatially variable genes (FDR <
0.05)
hs_genes = hs_results.loc[hs_results.FDR <
0.05].index

# Compute local correlations between genes
hs.compute_local_correlations(hs_genes,
jobs=8)

# Identify gene modules
modules = hs.create_modules(
    min_gene_threshold=10,  # Min genes per
module
    core_only=False,
    fdr_threshold=0.05
)

# Results: modules.Cluster contains module
assignments
```

Visualizing Hotspot Modules

- Module scores: aggregate expression of module genes
- Spatial plots: show where each module is active
- Heatmap: gene × spot, ordered by module
- Gene lists: functional enrichment (GO, KEGG)
- Compare modules to known cell types and pathways
- Validate with known markers

Code:

```
# Compute module scores
for module_id in modules.Cluster.unique():
    if module_id == -1:
        continue
    module_genes = modules[
        modules.Cluster == module_id
    ].index

    sc.tl.score_genes(
        adata,
        module_genes,
        score_name=f'module_{module_id}'
    )

# Plot module on tissue
sc.pl.spatial(
    adata,
    color=f'module_{module_id}',
    title=f'Module {module_id}'
)

# Gene set enrichment
from gseapy import enrichr
enr = enrichr(
    gene_list=module_genes,
    gene_sets='GO_Biological_Process'
)
```

Hotspot Best Practices & Tips

- Start with well-QC'd data (filter low-quality spots/genes)
- Choose n_neighbors based on tissue structure:
  - Small values (10-20): fine-grained patterns
  - Large values (30-50): broad regional patterns
- Computational efficiency:
  - Filter to highly variable genes before local correlations
  - Use latent_obsm_key='X_pca' for speed
- Validate modules with functional enrichment
- Compare with known biology and cell type markers

Neighborhood & Co-occurrence Analysis

Who Lives Next to Whom?

Why neighborhood matter?

Key questions:

1. Which cell types are neighbors?
2. Are certain cell type pairs enriched (appear together more than expected)?
3. Are certain pairs depleted (avoid each other)?
4. How far apart are different cell types?

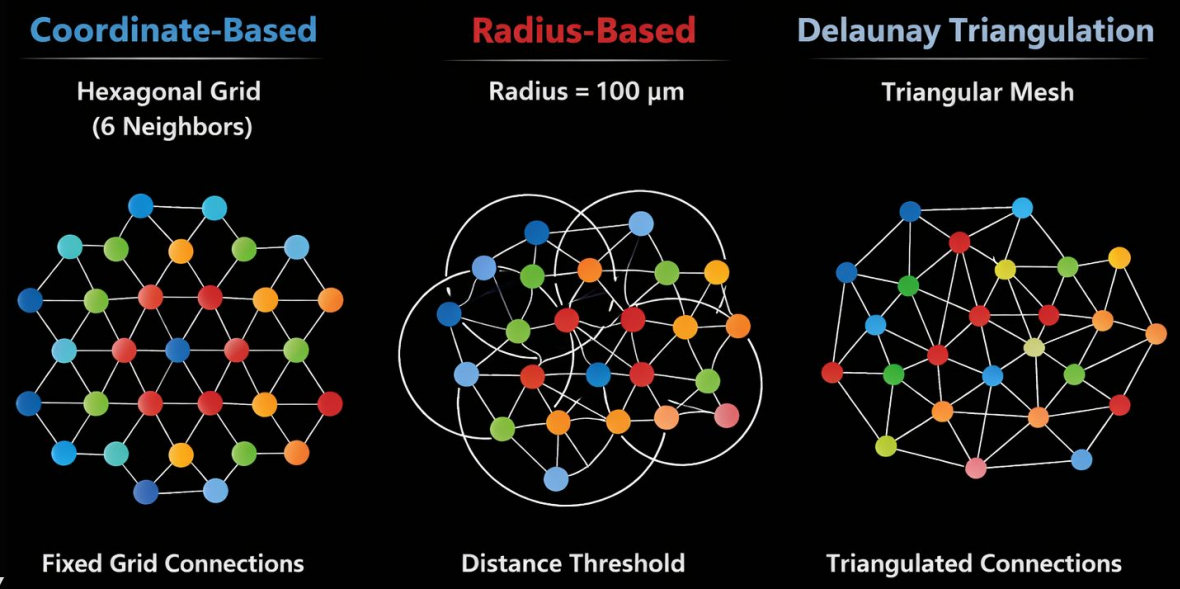
Why this matters:

- Cell-cell interactions require proximity
- Tissue organization reflects function
- Disease states show altered neighborhood patterns
- Example: In healthy tissue, immune cells patrol evenly. In tumors, they cluster at the invasive margin.

Step 1: Constructing Spatial Neighbor Graphs

Different strategies for defining spatial neighborhoods

```
import squidpy as sq
import scanpy as sc
# Load data with cell type annotations
adata = sc.read_h5ad('spatial_data_annotated.h5ad')
# Method 1: Coordinate-based
sq.gr.spatial_neighbors(
    adata,
    coord_type='grid',
    n_neighs=6  # Hexagonal grid: 6 neighbors
)
# Method 2: Generic spatial (any technology)
sq.gr.spatial_neighbors(adata,coord_type='generic',
    spatial_key='spatial',  # Key in adata.obsm
    radius=100,  # Distance threshold (microns))
# Method 3: Delaunay triangulation
sq.gr.spatial_neighbors(adata, coord_type='generic',
delaunay=True  # Connect nearest neighbors via triangulation
)
# Results stored in adata.obsp['spatial_connectivities']
# and adata.obsp['spatial_distances']
```



Cell Type Co-occurrence Analysis

- Question: Do certain cell types appear together more/less than expected?
- Method: Compare observed vs expected co-occurrence
- Expected = random permutation of cell type labels
- Statistical testing via permutations (typically 1000+)
- Results: Co-occurrence matrix with enrichment/depletion scores
- Positive = enrichment (prefer to be neighbors)
- Negative = depletion (avoid each other)
- Significant pairs reveal spatial organization principles

Computing Co-occurrence with Squidpy

Full workflow for co-occurrence analysis

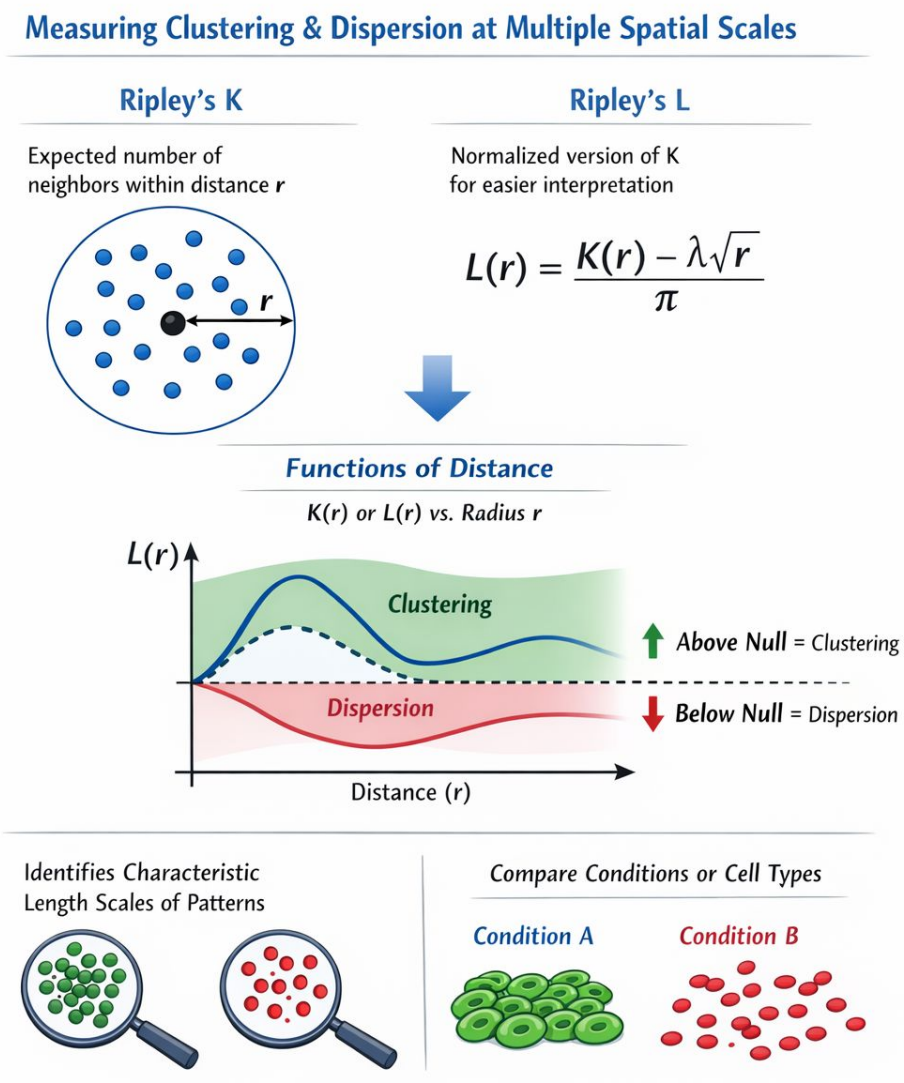
```
# Compute co-occurrence
sq.gr.co_occurrence(adata,
    cluster_key='cell_type',  # Column with cell type labels
    n_perms=1000,  # Number of permutations for significance
    copy=False
)
# Results stored in adata.uns['cell_type_co_occurrence']
# Contains: 'occ' (observed), 'exp' (expected), 'zscore', 'pvalue'
# Plot co-occurrence heatmap
sq.pl.co_occurrence(adata,cluster_key='cell_type',
    figsize=(8, 8), cmap='RdBu_r',  # Red = depleted, Blue = enriched
    vmin=-3, vmax=3
)
# Extract significant pairs
results = adata.uns['cell_type_co_occurrence']
zscore_df = results['zscore']
pvalue_df = results['pvalue']

# Filter for significant enrichment (z > 2, p < 0.05)
significant = (zscore_df > 2) & (pvalue_df < 0.05)
```

Neighborhood Enrichment Analysis

- Related to co-occurrence but different question:
  - Co-occurrence: Are cell types A and B neighbors?
  - Enrichment: Looking at A's neighborhood, is B over-represented?
- Counts cell type composition in each cell's neighborhood
- Tests if composition deviates from global distribution
- Useful for identifying cellular niches
- Output: Enrichment score per cell type pair
- Interpretation similar to co-occurrence

Ripley's Spatial Statistics

- Measures clustering/dispersion at multiple spatial scales
- Ripley's K: Expected number of neighbors within distance r
- Ripley's L: Normalized version of K for easier interpretation
- Functions of distance: K(r) or L(r) vs radius r
- Compare observed to random null distribution
- Above null = clustering, Below null = dispersion
- Identifies characteristic length scales of patterns
- Useful for comparing conditions or cell types



Computing Ripley's Functions

Ripley's functions with visualization

```
import squidpy as sq
import matplotlib.pyplot as plt

# Compute Ripley's L function for a specific cell type
sq.gr.ripley(
    adata,
    cluster_key='cell_type',
    mode='L',  # Use L function (or 'K' for K function)
    spatial_key='spatial',
    max_dist=500,  # Maximum distance to test (microns)
    n_steps=50,  # Number of distance intervals
    n_simulations=50,  # Monte Carlo simulations for CI
    copy=False
)
# Results in adata.uns['cell_type_ripley_L']
```

Spatial Domains & Communities

Segmenting Tissue into Regions

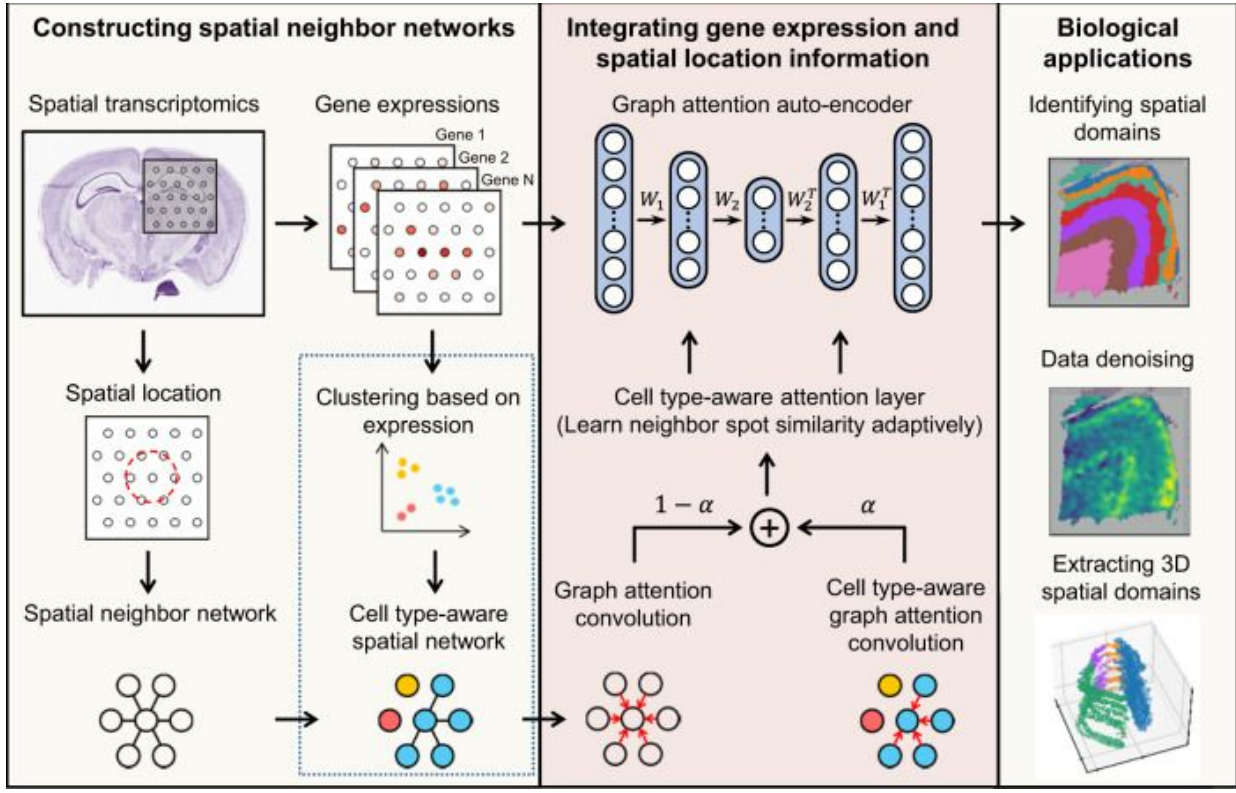
What are Spatial Domains?

- Tissue regions with distinct molecular signatures
- Combine two types of information:
  - Gene expression similarity (transcriptomic)
  - Spatial proximity (location)
- Different from clustering alone (which ignores space)
- Domains should be:
  - Molecularly distinct (different expression profiles)
  - Spatially contiguous (connected regions)
  - Biologically interpretable (match known structures)
- Applications: tissue atlas, disease staging, functional regions

STAGATE: Spatial Domain Detection

Graph attention network for spatial transcriptomics

- Integrates gene expression with spatial information
- Graph attention mechanism learns spatial relationships
- Denoises data while preserving spatial structure
- Outputs: low-dimensional embedding + domain labels
- State-of-the-art performance on multiple datasets
- Works for various spatial technologies



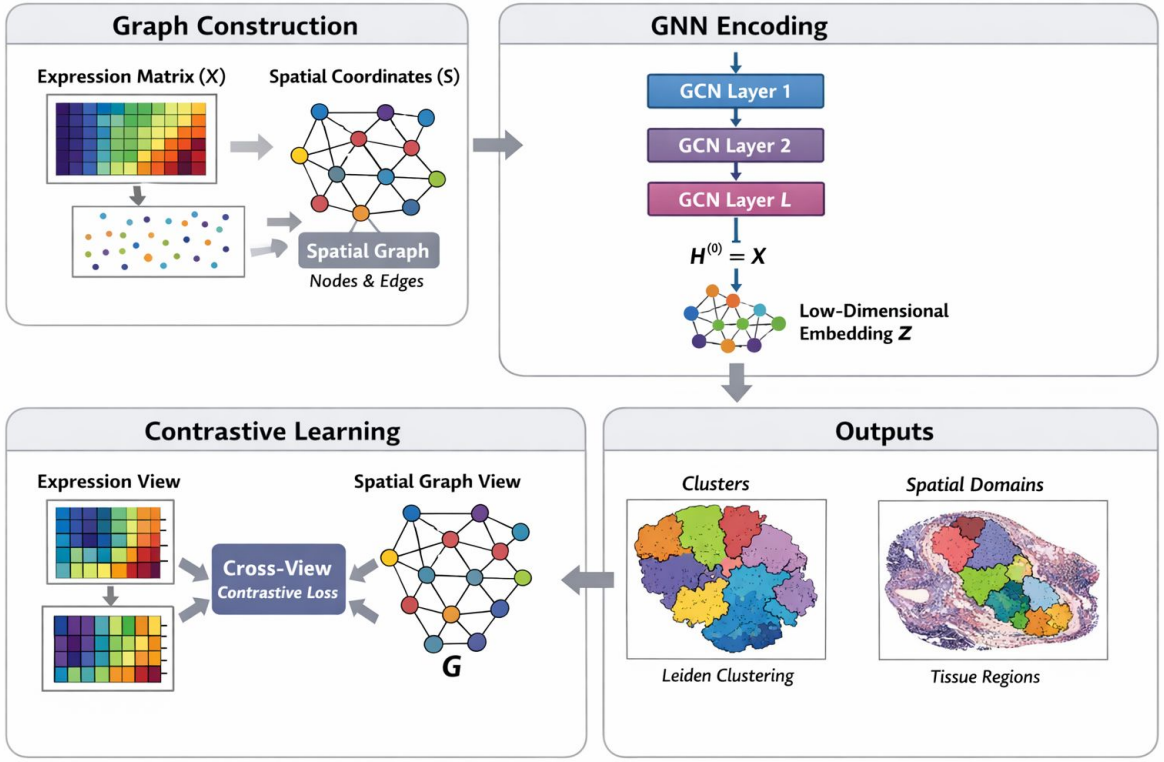
STAGATE Workflow: Domain Detection

Complete STAGATE pipeline

```
import scanpy as sc
import STAGATE

# Load and preprocess data
adata = sc.read_h5ad('spatial_data.h5ad')
sc.pp.normalize_total(adata)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, n_top_genes=3000)

# Construct spatial graph
# For Visium:
STAGATE.Cal_Spatial_Net(adata, rad_cutoff=150)
# For other technologies: adjust rad_cutoff based on resolution

# Train STAGATE model
adata = STAGATE.train_STAGATE(
    adata,
    alpha=0,  # Weight for reconstruction loss (0=use default)
    n_epochs=1000,  # Training iterations
    device='cuda:0'  # Use GPU (or 'cpu' for CPU)
)

# STAGATE outputs embedding in adata.obsm['STAGATE']
```

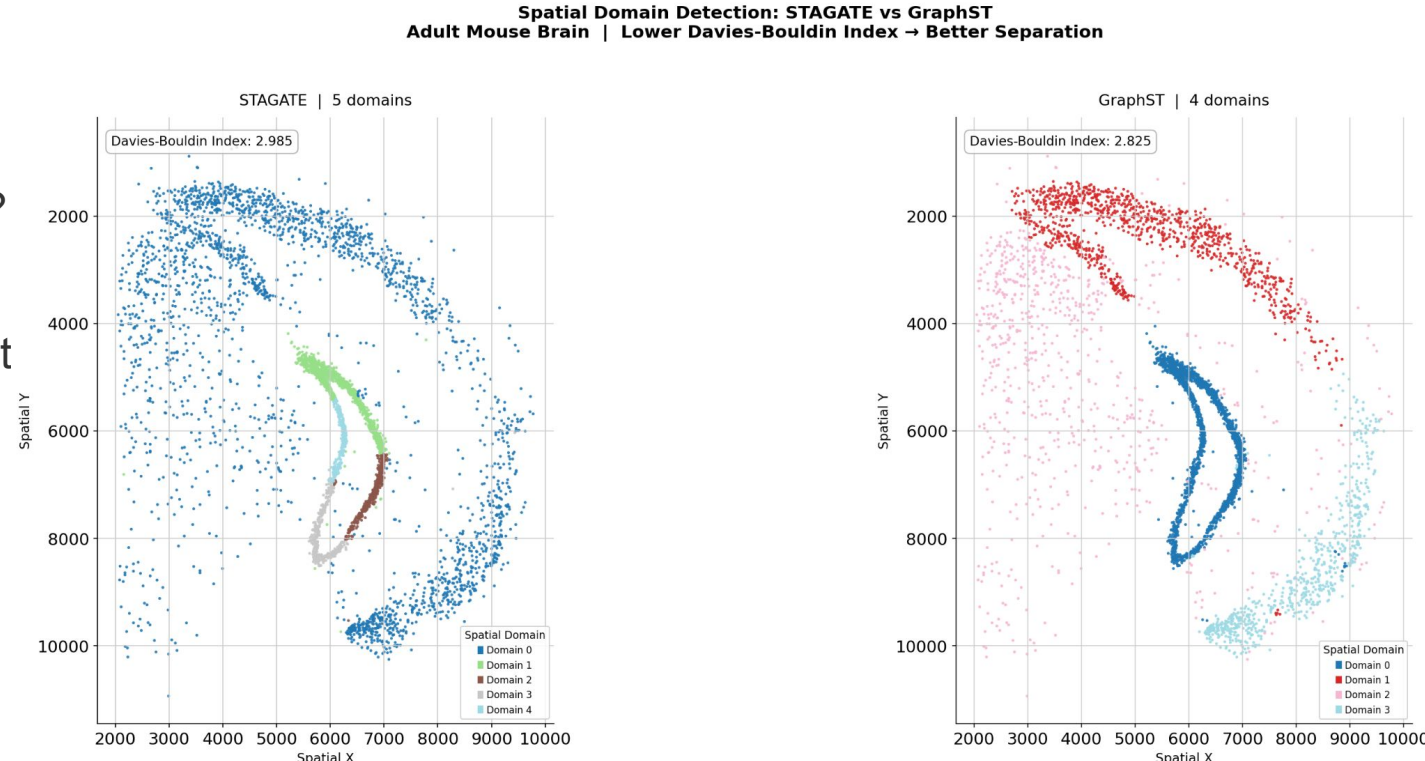
GraphST

- Another graph-based deep learning approach
- Uses graph self-supervised contrastive learning
- Applies a Graph Neural Network (GNN) to learn representations that preserve transcriptional similarity
- Available in Python, easy to install



Interpreting and Validating Spatial Domains

- Step 1: Visualize domains on tissue
- Step 2: Find marker genes per domain (differential expression)
- Step 3: Functional enrichment of domain markers
- Step 4: Compare to known anatomy/histology
- Step 5: Validate in independent samples
- Questions to ask:
  - Do domains match expected tissue structures?
  - Are domain boundaries biologically meaningful?
  - Do domains replicate across samples?
  - Can you name domains based on markers/funct



Ligand-Receptor Interaction Analysis

Decoding Cell-Cell Communication

Principles of Cell-Cell Communication

- Cells communicate via ligand-receptor (L-R) pairs
- Ligand: Signaling molecule (secreted or membrane-bound)
- Receptor: Protein that detects ligand (on cell surface)
- Communication requires:
  - Ligand expression in sender cell
  - Receptor expression in receiver cell
  - Spatial proximity (cells must be neighbors)
- Database of known L-R pairs (CellPhoneDB, CellChatDB, etc.)
- Signaling pathways organize L-R pairs by function

CellPhoneDB: L-R Database and Method

- Comprehensive database of ligand-receptor interactions
- Statistical method for identifying active L-R pairs
- Tests if ligand + receptor are co-expressed above chance:
  - For each cell type pair (A→B) and L-R pair:
    - - Check if cell type A expresses ligand
    - - Check if cell type B expresses receptor
    - - Compute mean expression product: mean(ligand_A) × mean(receptor_B)
  - Permute cell type labels many times (e.g., 1000)
  - Recompute mean products under permutations
  - Compare observed to null distribution
  - Compute p-value: fraction of permutations with higher mean product
- Squidpy extension: adds spatial constraint
  - Standard CellPhoneDB: Any cell type pair
  - Spatial CellPhoneDB: Only neighboring cell type pairs
- Output: Matrix of significant interactions per cell type pair

Spatial Ligand-Receptor Analysis with Squidpy

Complete spatial L-R workflow

```
import squidpy as sq
import scanpy as sc

# Assume: data with cell types and spatial graph already built

# Run spatial ligand-receptor analysis
sq.gr.ligrec(
    adata,
    n_perms=1000,  # Permutations for significance
    cluster_key='cell_type',  # Cell type annotations
    copy=False,
    use_raw=False,  # Use normalized data
    transmitter_params={'categories': 'ligand'},
    receiver_params={'categories': 'receptor'},
)

# Results in adata.uns['cell_type_ligrec']

# Extract significant interactions
ligrec_res = adata.uns['cell_type_ligrec']
significant = ligrec_res[ligrec_res['pvalue'] < 0.05].copy()

# Focus on specific cell type pair
tumor_tcell = significant[
```

Interpreting Ligand-Receptor Results

- Prioritize interactions by:
  - Statistical significance (p-value < 0.05)
  - Effect size (high mean expression product)
  - Biological relevance (known important pathways)
- Group interactions by signaling pathway
- Compare interactions across conditions (disease vs healthy)
- Validate spatially: plot ligand and receptor expression
- Consider both directions: A→B and B→A
- Generate hypotheses for experimental validation

Advanced Spatial Patterns

Gradients and Trajectories

Detecting Spatial Gradients

- Gradients: Smooth continuous changes in expression
- Examples:
  - Oxygen gradient from blood vessels
  - Morphogen gradients in development
  - Metabolic zonation (liver, kidney)
- Detection methods:
  - Distance from landmarks (vessels, boundaries)
  - Polynomial regression on coordinates
  - Principal curve fitting
- Squidpy: Use spatial coordinates as features for regression

Spatial Trajectory Analysis

- Spatial trajectories: Ordered paths through tissue
- Different from pseudotime (which ignores space)
- Applications:
  - Cancer progression: core → invasive edge
  - Immune infiltration: blood → tumor
  - Developmental axes: anterior-posterior
  - Differentiation gradients in tissue
- Methods: Combine spatial + transcriptomic information
- Tools: Palantir, PAGA with spatial constraints

Summary & Key Takeaways

Wrapping Up Lesson 3

Spatial Analysis Methods: Quick Reference

Gene/Module-Focused

- Moran's I: Fast, scalable SVG detection
- SpatialDE: Model-based, pattern characterization
- Hotspot: Spatial gene modules, coherent patterns
- Getis-Ord: Local hotspot identification
- Use for: Identifying spatial expression patterns

Cell/Tissue-Focused

- Squidpy: Neighborhoods, co-occurrence, L-R
- STAGATE/GraphST: Spatial domain detection
- Ripley's functions: Multi-scale clustering
- CellPhoneDB: Cell communication networks
- Use for: Understanding tissue organization

Lesson Summary - What We've Learned

- Spatial analysis reveals tissue organization beyond gene expression
- Multiple complementary approaches needed:
  - Moran's I for spatially variable genes
  - Hotspot for gene modules with spatial coherence
  - Squidpy for neighborhood and co-occurrence
  - STAGATE/GraphST for spatial domain detection
  - Ligand-receptor analysis for cell communication
- Integration provides comprehensive tissue understanding
- Always validate findings with biological knowledge!

Tools & Resources Cheat Sheet

- Squidpy: squidpy.readthedocs.io (neighborhoods, enrichment, L/R)
- Hotspot: github.com/yoseflab/Hotspot (gene modules)
- SpatialDE: github.com/Teichlab/SpatialDE (variable genes)
- STAGATE: stagate.readthedocs.io (spatial domains)
- Scanpy: scanpy.readthedocs.io (data processing foundation)
- Installation: pip install squidpy scanpy hotspot-sc

Next Steps: Continue Your Spatial Journey

- Hands-on exercises:
  - Exercise 1: Moran's I on example dataset
  - Exercise 2: Hotspot gene modules
  - Exercise 3: Squidpy neighborhood analysis
  - Exercise 4: STAGATE spatial domains
  - Exercise 5: Ligand-receptor interactions
- Advanced topics (Lesson 4):
  - Multi-sample integration

Key Takeaways: What You Should Remember

- 1. Space matters: Tissue context reveals biology invisible to scRNA-seq
- 2. Multiple complementary methods needed (SVGs, modules, domains, L-R)
- 3. Integration provides comprehensive tissue understanding
- 4. Always validate spatially: plot everything on tissue!
- 5. Statistics guide, biology interprets: p-values ≠ truth
- 6. Tools are accessible: Squidpy + Scanpy = 90% of analyses
- 7. Reproducibility essential: share code and data
- 8. Future is bright: field is young, lots of room for innovation!

Questions?

Thank you for your attention!